Versión 0.3
Cómo construir *paso a paso* la tabla y la curva ROC (y PR) con desgloses completos y script en Python

Prof. Alberto Muñoz / Tec de Monterrey

Abstract

Este documento presenta una guía breve y rigurosa para construir, interpretar y graficar la curva ROC (Receiver Operating Characteristic) a partir de un conjunto mínimo de datos. La explicación es auto-contenida y prescinde de dependencias complejas, de modo que pueda incorporarse como material de arranque en cursos introductorios de aprendizaje automático, visión por computadora y control de calidad. Incluimos la tabla ROC paso a paso, la curva correspondiente y un script en `Python` estándar (sin librerías especializadas), junto con observaciones prácticas sobre empates, discretización por tamaño de muestra y el cálculo del AUC por regla del trapecio.

Motivación y contexto

Mi motivación para elaborar esta explicación surge de mi experiencia previa preparando y cursando la **certificación del curso Computer Vision for Industrial Inspection de NVIDIA**. En ese proceso identifiqué que, aun cuando el temario cubre con solidez los componentes de entrenamiento, optimización y despliegue, *la lectura fina de métricas de clasificación* (en particular la construcción manual de la curva ROC y la interpretación de sus escalones, diagonales y áreas) suele darse por sentada o tratarse de manera acelerada.

Considero que este tema debe abordarse explícitamente desde la introducción del curso por tres razones:

1. **Criterios operativos claros:** Al entender cómo se obtiene cada punto (fp, tp) al mover el umbral, el estudiante puede alinear la métrica con el objetivo del sistema (por ejemplo, priorizar *recall* en inspección de defectos vs. priorizar *precision*).

2. **Tomas de decisión reproducibles:** La construcción manual de la tabla ROC revela el efecto de los empates de puntuación y del tamaño de muestra, evitando interpretaciones erróneas cuando se comparan modelos o umbrales.

3. **Puente hacia despliegue:** Comprender ROC/AUC facilita la traducción de requisitos del dominio (costos de FP/FN, umbrales regulados, *specs* de SLA) a configuraciones concretas de inferencia y monitoreo en producción.

Mi objetivo es que el alumno **vea y replique** el mecanismo de construcción de la **tabla ROC** y de la **curva ROC**, y que además pueda contrastar con **Precision–Recall (PR)**. Para ello, usamos un umbral t y definimos la predicción binaria

$$\hat{y} = \mathbf{1}[\text{score} \geq t].$$

A partir de $\hat{y}$ y de la etiqueta verdadera $y \in \{0, 1\}$, contamos:

$$\text{TP, FP, TN, FN} \quad \Rightarrow \quad \text{TPR} = \frac{\text{TP}}{P},\ \text{FPR} = \frac{\text{FP}}{N},\ \text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}},\ \text{NPV} = \frac{\text{TN}}{\text{TN}+\text{FN}},$$

En este material, por tanto, presento una derivación *paso a paso* con datos mínimos y código elemental, de forma que cualquier participante del curso **certificación del curso Computer Vision for Industrial Inspection de NVIDIA** (o cursos afines) pueda:

- reconstruir la tabla ROC con papel y lápiz,
- generar la curva con `TikZ` o con un script básico,
- y calcular un AUC consistente con la regla del trapecio sin depender de bibliotecas externas.

Esta sección cumple la función de **prerrequisito conceptual** para los módulos posteriores (entrenamiento, calibración de umbrales, *post-processing* y despliegue), asegurando que las decisiones métricas estén justificadas y sean auditables desde el inicio.

1 Introducción práctica: control de calidad en envases de vidrio

1.1 Contexto

Imagina una línea de producción de **botellas de vidrio** para bebidas. Un defecto crítico son las *microfisuras* que pueden provocar fugas o roturas. La planta desea automatizar la inspección usando **visión por computadora**: cada botella es fotografiada y un modelo produce un *score* continuo (mayor = más sospecha de defecto).

1.2 Cómo se generan los *datos iniciales*

Para construir y evaluar la curva ROC/PR usaremos un conjunto didáctico[1] con 20 botellas (10 sanas y 10 defectuosas) y seguiremos estos pasos estándar en QA/QC:

1) Muestreo y *etiqueta de referencia* (ground truth). Seleccionamos 20 botellas: 10 confirmadas **defectuosas** mediante una *prueba hidrostática* (presión interna controlada + inspección con lupa) y 10 **sanas** certificadas por un inspector senior.

2) Extracción de señales y *score* del modelo. Cada botella se pasa por un *pipeline* de visión por computadora (iluminación anular + cámara de alta resolución):

- *Segmentación* del contorno y zonas críticas (cuello, rosca, cuerpo).
- *Detección* de patrones de grietas: gradientes anómalos, texturas de alta frecuencia, *line segments*.
- Un clasificador produce un **score continuo** $s \in \mathbb{R}$ (compatibilidad con "defectuosa").

3) Normalización y calibración del score. Para que los scores sean comparables entre turnos y cámaras:

1. Normalizamos por iluminación y escala geométrica.
2. Aplicamos una **calibración probabilística** (p.ej., *Platt scaling* o *isotonic regression*) sobre un lote histórico, de modo que s pueda interpretarse como *probabilidad calibrada* de defecto.

4) Datos de ejemplo (listados N y P). Tras calibrar, este *lote didáctico* produce los siguientes *scores*[2]: $\mathbf{N} = \{0.3, 0.4, 0.5, 0.5, 0.5, 0.6, 0.7, 0.7, 0.8, 0.9\}$, $\mathbf{P} = \{0.5, 0.6, 0.6, 0.8, 0.9, 0.9, 0.9, 1.0, 1.2, 1.4\}$.

Aquí $\mathbf{N}$ son las botellas *sanas* ($y = 0$) y $\mathbf{P}$ las *defectuosas* ($y = 1$). El valor **1.4** es simplemente el **máximo score observado** en el lote (primera botella detectada al bajar el umbral).

[1] Por ejemplo, consideramos el mismo número de positivos y negativos, cosa que no siempre sucede

[2] Estos datos son exactamente los mismos que los utilizados en el ejemplo ilustrativo del libro citado al final del documento

1.3 Política de umbral inicial y matriz de confusión

En control de calidad, **fallar con un defecto que se escapa** (falso negativo, FN) suele ser más costoso que revisar manualmente una botella sana marcada por error (falso positivo, FP). Por ello, una *política inicial razonable* es usar un umbral relativamente **permisivo** para *atrapar la mayoría de los defectos.*

A modo de **punto de partida**, tomemos $t = 0.6$. Con $P = N = 10$ y los datos anteriores, la cuenta es:

$$\text{TP} = 9, \quad \text{FP} = 5, \quad \text{TN} = 5, \quad \text{FN} = 1.$$

Las métricas derivadas son:

$$\text{TPR} = \tfrac{9}{10} = 0.90, \quad \text{FPR} = \tfrac{5}{10} = 0.50, \quad \text{Precisión} = \tfrac{9}{9+5} = 0.643, \quad \text{NPV} = \tfrac{5}{5+1} = 0.833.$$

Table 1: Matriz de confusión inicial para $t = 0.6$ (priorizando capturar defectos).

	Pred. Negativa	**Pred. Positiva**
Real Negativa (N=10)	TN = 5	FP = 5
Real Positiva (P=10)	FN = 1	TP = 9

Discusión operativa.

- Con $t = 0.6$ *perdemos* sólo 1 defectuosa (FN=1), a costa de revisar 5 sanas marcadas (FP=5).
- Si la planta quisiera **menos revisiones extra**, podría subir t (p.ej., $t = 0.8$: TP=7, FP=2), pero aumentaría el riesgo de que *se escape* un defecto (FN=3).
- Esta tensión FN/FP es la que la **curva ROC** resume al barrer todos los t: de ahí su utilidad para decidir políticas.

2 Datos del ejemplo y motivación didáctica

Tenemos dos grupos de 10 individuos con *scores* (a mayor valor, mayor probabilidad de ser positivo real): $\mathbf{N} = \{0.3, 0.4, 0.5, 0.5, 0.5, 0.6, 0.7, 0.7, 0.8, 0.9\}$, $\mathbf{P} = \{0.5, 0.6, 0.6, 0.8, 0.9, 0.9, 0.9, 1.0, 1.2, 1.4\}$.

Veamos el mecanismo de construcción de la **tabla ROC** y de la **curva ROC**, y contrastar con **Precision–Recall (PR)**. Para ello, usamos las definiciones anteriores como el umbral t y definimos la predicción binaria

$$\hat{y} = \mathbf{1}[\text{score} \geq t].$$

A partir de $\hat{y}$ y de la etiqueta verdadera $y \in \{0, 1\}$, contamos:

$$\text{TP, FP, TN, FN} \quad \Rightarrow \quad \text{TPR} = \frac{\text{TP}}{P}, \ \text{FPR} = \frac{\text{FP}}{N}, \ \text{Precision} = \frac{\text{TP}}{\text{TP+FP}}, \ \text{NPV} = \frac{\text{TN}}{\text{TN+FN}},$$

donde para este ejemplo $P = \#\mathbf{P} = 10$ y $N = \#\mathbf{N} = 10$.

3 ¿De dónde sale el 1.4?

El número 1.4 **sale de los datos**: es el *mayor score* observado en todo el conjunto.

$$\max(\mathbf{N} \cup \mathbf{P}) = \max(\{0.3, \ldots, 0.9,\ 0.5, \ldots, 1.4\}) = 1.4.$$

Por construcción de ROC:

1. Primero ponemos $t > \max = 1.4$. Nadie supera el umbral $\Rightarrow$ TP = FP = 0 y el punto inicial de la curva es $(\mathrm{FPR}, \mathrm{TPR}) = (0, 0)$.
2. Luego bajamos a $t = 1.4$. Exactamente *un* positivo (el de score 1.4) cruza el umbral: $\mathrm{TP} = 1,\ \mathrm{FP} = 0 \Rightarrow (\mathrm{FPR}, \mathrm{TPR}) = (0.0, 0.1)$.

4 Cómputo *paso a paso* de TP, FP, TN, FN, TPR, FPR, Precisión y NPV

A continuación desglosamos **todos** los umbrales (incluyendo el origen $t > \max$). Tomamos $P = N = 10$.

Origen: $t > 1.4$ (ligeramente mayor que el máximo)

Nadie alcanza el umbral:

$$\mathrm{TP} = 0,\ \mathrm{FP} = 0,\ \mathrm{TN} = 10,\ \mathrm{FN} = 10.$$

	Pred. +	**Pred. -**
Actual +	TP = 0	FN = 10
Actual -	FP = 0	TN = 10

$$\mathrm{TPR} = \frac{0}{10} = 0.000, \quad \mathrm{FPR} = \frac{0}{10} = 0.000, \quad \mathrm{Precision} = \frac{0}{0}\ (\text{convención}) = 0.000, \quad \mathrm{NPV} = \frac{10}{10+10} = 0$$

$t = 1.4$

Positivos ≥ 1.4: $\{1.4\} \Rightarrow \mathrm{TP} = 1$.
Negativos ≥ 1.4: ninguno $\Rightarrow \mathrm{FP} = 0$.

$$\mathrm{TN} = 10,\ \mathrm{FN} = 9.$$

	Pred. +	**Pred. -**
Actual +	TP = 1	FN = 9
Actual -	FP = 0	TN = 10

$$\mathrm{TPR} = \tfrac{1}{10} = 0.100,\ \mathrm{FPR} = \tfrac{0}{10} = 0.000,\ \mathrm{Precision} = \tfrac{1}{1+0} = 1.000,\ \mathrm{NPV} = \tfrac{10}{10+9} = 0.526.$$

$t = 1.2$

Positivos ≥ 1.2: $\{1.2, 1.4\} \Rightarrow \text{TP} = 2$.
Negativos ≥ 1.2: ninguno $\Rightarrow \text{FP} = 0$.

$$\text{TN} = 10,\ \text{FN} = 8.$$

	Pred. +	**Pred. -**
Actual +	TP = 2	FN = 8
Actual -	FP = 0	TN = 10

$$\text{TPR} = \frac{2}{10} = 0.200,\ \text{FPR} = \frac{0}{10} = 0.000,\ \text{Precision} = \frac{2}{2+0} = 1.000,\ \text{NPV} = \frac{10}{10+8} = 0.556.$$

$t = 1.0$

Positivos ≥ 1.0: $\{1.0, 1.2, 1.4\} \Rightarrow \text{TP} = 3$.
Negativos ≥ 1.0: ninguno $\Rightarrow \text{FP} = 0$.

$$\text{TN} = 10,\ \text{FN} = 7.$$

	Pred. +	**Pred. -**
Actual +	TP = 3	FN = 7
Actual -	FP = 0	TN = 10

$$\text{TPR} = \frac{3}{10} = 0.300,\ \text{FPR} = \frac{0}{10} = 0.000,\ \text{Precision} = \frac{3}{3+0} = 1.000,\ \text{NPV} = \frac{10}{10+7} = 0.588.$$

$t = 0.9$

Positivos ≥ 0.9: $\{0.9, 0.9, 0.9, 1.0, 1.2, 1.4\} \Rightarrow \text{TP} = 6$.
Negativos ≥ 0.9: $\{0.9\} \Rightarrow \text{FP} = 1$.

$$\text{TN} = 9,\ \text{FN} = 4.$$

	Pred. +	**Pred. -**
Actual +	TP = 6	FN = 4
Actual -	FP = 1	TN = 9

$$\text{TPR} = \frac{6}{10} = 0.600,\ \text{FPR} = \frac{1}{10} = 0.100,\ \text{Precision} = \frac{6}{6+1} = 0.857,\ \text{NPV} = \frac{9}{9+4} = 0.692.$$

$t = 0.8$

Positivos ≥ 0.8: $\{0.8, 0.9, 0.9, 0.9, 1.0, 1.2, 1.4\} \Rightarrow \text{TP} = 7$.
Negativos ≥ 0.8: $\{0.8, 0.9\} \Rightarrow \text{FP} = 2$.

$$\text{TN} = 8,\ \text{FN} = 3.$$

	Pred. +	**Pred. -**
Actual +	TP = 7	FN = 3
Actual -	FP = 2	TN = 8

$$\text{TPR} = \frac{7}{10} = 0.700,\ \text{FPR} = \frac{2}{10} = 0.200,\ \text{Precision} = \frac{7}{7+2} = 0.778,\ \text{NPV} = \frac{8}{8+3} = 0.727.$$

$t = 0.7$

Positivos ≥ 0.7: $\{0.8, 0.9, 0.9, 0.9, 1.0, 1.2, 1.4\} \Rightarrow \text{TP} = 7$ (no hay 0.7 en $\mathbf{P}$).
Negativos ≥ 0.7: $\{0.7, 0.7, 0.8, 0.9\} \Rightarrow \text{FP} = 4$.

$$\text{TN} = 6,\ \text{FN} = 3.$$

	Pred. +	**Pred. -**
Actual +	TP = 7	FN = 3
Actual -	FP = 4	TN = 6

$$\text{TPR} = \tfrac{7}{10} = 0.700,\ \text{FPR} = \tfrac{4}{10} = 0.400,\ \text{Precision} = \tfrac{7}{7+4} = 0.636,\ \text{NPV} = \tfrac{6}{6+3} = 0.667.$$

$t = 0.6$

Positivos ≥ 0.6: $\{0.6, 0.6, 0.8, 0.9, 0.9, 0.9, 1.0, 1.2, 1.4\} \Rightarrow \text{TP} = 9$.
Negativos ≥ 0.6: $\{0.6, 0.7, 0.7, 0.8, 0.9\} \Rightarrow \text{FP} = 5$.

$$\text{TN} = 5,\ \text{FN} = 1.$$

	Pred. +	**Pred. -**
Actual +	TP = 9	FN = 1
Actual -	FP = 5	TN = 5

$$\text{TPR} = \tfrac{9}{10} = 0.900,\ \text{FPR} = \tfrac{5}{10} = 0.500,\ \text{Precision} = \tfrac{9}{9+5} = 0.643,\ \text{NPV} = \tfrac{5}{5+1} = 0.833.$$

$t = 0.5$

Positivos ≥ 0.5: los 10 en $\mathbf{P} \Rightarrow \text{TP} = 10$.
Negativos ≥ 0.5: $\{0.5, 0.5, 0.5, 0.6, 0.7, 0.7, 0.8, 0.9\} \Rightarrow \text{FP} = 8$.

$$\text{TN} = 2,\ \text{FN} = 0.$$

	Pred. +	**Pred. -**
Actual +	TP = 10	FN = 0
Actual -	FP = 8	TN = 2

$$\text{TPR} = \tfrac{10}{10} = 1.000,\ \text{FPR} = \tfrac{8}{10} = 0.800,\ \text{Precision} = \tfrac{10}{10+8} = 0.556,\ \text{NPV} = \tfrac{2}{2+0} = 1.000.$$

$t = 0.4$

Positivos ≥ 0.4: $10 \Rightarrow \text{TP} = 10$.
Negativos ≥ 0.4: $\{0.4, 0.5, 0.5, 0.5, 0.6, 0.7, 0.7, 0.8, 0.9\} \Rightarrow \text{FP} = 9$.

$$\text{TN} = 1,\ \text{FN} = 0.$$

	Pred. +	**Pred. -**
Actual +	TP = 10	FN = 0
Actual -	FP = 9	TN = 1

$$\text{TPR} = 1.000,\ \text{FPR} = 0.900,\ \text{Precision} = \tfrac{10}{10+9} = 0.526,\ \text{NPV} = \tfrac{1}{1+0} = 1.000.$$

$t = 0.3$

Positivos ≥ 0.3: $10 \Rightarrow \text{TP} = 10$.
Negativos ≥ 0.3: los $10 \Rightarrow \text{FP} = 10$.

$$\text{TN} = 0, \ \text{FN} = 0.$$

	Pred. +	**Pred. -**
Actual +	TP = 10	FN = 0
Actual -	FP = 10	TN = 0

$$\text{TPR} = 1.000, \ \text{FPR} = 1.000, \ \text{Precision} = \tfrac{10}{10+10} = 0.500, \ \text{NPV} = \tfrac{0}{0+0} = 0.000.$$

(*Opcional*: $t < 0.3$ produce exactamente la misma fila; por eso suele omitirse.)

5 Tabla ROC resultante (con métricas completas)

t	TP	FP	TN	FN	TPR	FPR	Precisión	NPV
> 1.4	0	0	10	10	0.000	0.000	0.000	0.500
1.4	1	0	10	9	0.100	0.000	1.000	0.526
1.2	2	0	10	8	0.200	0.000	1.000	0.556
1.0	3	0	10	7	0.300	0.000	1.000	0.588
0.9	6	1	9	4	0.600	0.100	0.857	0.692
0.8	7	2	8	3	0.700	0.200	0.778	0.727
0.7	7	4	6	3	0.700	0.400	0.636	0.667
0.6	9	5	5	1	0.900	0.500	0.643	0.833
0.5	10	8	2	0	1.000	0.800	0.556	1.000
0.4	10	9	1	0	1.000	0.900	0.526	1.000
0.3	10	10	0	0	1.000	1.000	0.500	0.000

Table 2: Tabla ROC completa para el ejemplo. $P = N = 10$. El valor 1.4 proviene del mayor score de los datos.

5.1 Apunte sobre PR (Precision–Recall)

Para cada umbral t, además de $(\text{FPR}(t), \text{TPR}(t))$ podemos formar $(\text{recall}(t), \text{precision}(t))$, donde recall = TPR y precision = PPV. En conjuntos desbalanceados, la curva PR y su área (AUPRC) son especialmente informativas.

5.2 Elección del mejor umbral y efecto de aumentarlo

En el contexto de inspección de calidad de botellas de vidrio, el documento describe un escenario con $P = N = 10$, donde cada botella (sana o defectuosa) recibe un puntaje continuo entre 0.3 y 1.4 según la probabilidad de defecto. El objetivo es seleccionar un umbral t que permita distinguir adecuadamente las botellas defectuosas de las sanas.

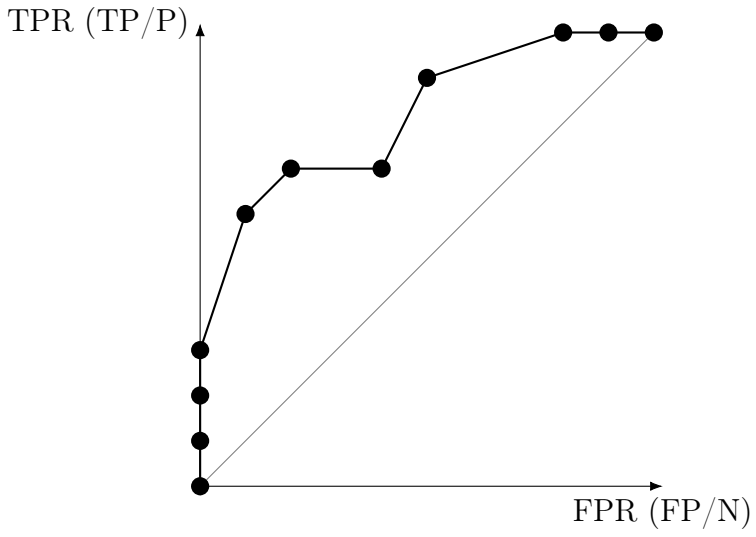

Figure 1: Curva ROC empírica y diagonal de azar.

5.3 Umbral óptimo según el contexto de control de calidad

De acuerdo con los resultados tabulados, el umbral $t = 0.6$ ofrece el **mejor compromiso** entre sensibilidad y esfuerzo operativo:

- Captura 9 de las 10 botellas defectuosas (TPR = 0.9),
- Sólo deja escapar 1 defecto (FN = 1),
- Y mantiene un nivel de falsos positivos moderado (FP = 5, FPR = 0.5).

En la práctica industrial, este valor se considera adecuado porque en control de calidad **es preferible detectar de más (FP)** que permitir la salida de un producto defectuoso (FN), cuyo costo es mucho más alto. Por tanto, el umbral $t = 0.6$ se recomienda como *política inicial* para priorizar la seguridad del proceso, garantizando una alta tasa de detección de defectos sin sobrecargar de manera excesiva la revisión manual.

5.4 Efecto de aumentar el umbral

Si se aumenta el umbral (por ejemplo, de $t = 0.6$ a $t = 0.8$ o $t = 1.0$), el clasificador se vuelve más estricto: sólo considera defectuosas a las botellas con puntajes muy altos.

- Disminuyen los **falsos positivos (FP)** — se marcan menos botellas sanas —,
- Pero aumentan los **falsos negativos (FN)** — más botellas defectuosas se escapan —.

Este fenómeno se refleja en la curva ROC como un desplazamiento hacia la parte inferior derecha, donde la tasa de verdaderos positivos (TPR) disminuye a medida que el umbral crece. Por ejemplo:

$$t = 0.6 \Rightarrow (\mathrm{TPR} = 0.9, \mathrm{FPR} = 0.5), \quad t = 0.8 \Rightarrow (\mathrm{TPR} = 0.7, \mathrm{FPR} = 0.2), \quad t = 1.0 \Rightarrow (\mathrm{TPR} = 0.3, \mathrm{FP}$$

Por tanto, **aumentar el umbral reduce la sensibilidad del sistema** y puede ser deseable sólo si el costo de una revisión innecesaria (FP) es significativamente mayor que el costo de dejar pasar un defecto (FN).

5.5 Reflexión

Con estos datos se concluye que, bajo los criterios del ejemplo de inspección de envases de vidrio, el umbral $t = 0.6$ es el punto de operación más equilibrado: mantiene alta la tasa de detección de defectos con un nivel aceptable de revisiones falsas, ofreciendo así la **mejor predicción global** en términos prácticos para el entorno industrial descrito.

6 Script en Python (sin NumPy) para reproducir tablas y gráficas

Cómo usar y qué esperar

El script:

- Genera la lista ordenada de umbrales (incluye max $+\varepsilon$ y min $-\varepsilon$).
- Para cada umbral, construye la matriz de confusión y calcula TPR, FPR, Precisión y NPV (tablas *legibles* en consola).
- Traza **ROC** y calcula $\mathrm{AUC}_{\mathrm{ROC}}$ por regla del trapecio.
- Traza **PR** y calcula AUPRC; añade la línea base de azar (tasa de positivos).

Código (con resaltado de sintaxis)

Listing 1: ROC y PR: tabla, AUC y GRÁFICAS (consola y ventana).

```python
# ============================
# ROC y PR: tabla, AUC y GR FICAS (en consola y ventana)
# ============================
# Requisitos: solo matplotlib (sin NumPy).
import matplotlib.pyplot as plt

# ---------- Utilidades base (sin NumPy) ----------
def build_thresholds(y_prob, add_endpoint_eps=True):
    vals = sorted(set(float(v) for v in y_prob), reverse=True)
    if add_endpoint_eps:
        mn, mx = min(y_prob), max(y_prob)
        eps = 1e-12
        vals = [mx + eps] + vals + [mn - eps]
    return vals

def confusion_counts(y_true, y_pred):
    TP = FP = TN = FN = 0
    for t, p in zip(y_true, y_pred):
        if p == 1 and t == 1: TP += 1
        elif p == 1 and t == 0: FP += 1
        elif p == 0 and t == 0: TN += 1
        elif p == 0 and t == 1: FN += 1
    return TP, FP, TN, FN

```

```python
def metrics_from_counts(TP, FP, TN, FN):
    P = TP + FN
    N = FP + TN
    TPR = (TP / P) if P else 0.0
    FPR = (FP / N) if N else 0.0
    TNR = (TN / N) if N else 0.0
    PPV = (TP / (TP + FP)) if (TP + FP) else 0.0
    NPV = (TN / (TN + FN)) if (TN + FN) else 0.0
    return TPR, FPR, TNR, PPV, NPV

# ---------- ROC: tabla y AUC ----------
def compute_roc_table(y_true, y_prob, thresholds=None, add_endpoint_eps=True):
    if thresholds is None:
        thresholds = build_thresholds(y_prob, add_endpoint_eps=add_endpoint_eps)

    rows = []
    P = sum(1 for t in y_true if t == 1)
    N = sum(1 for t in y_true if t == 0)

    for tau in thresholds:
        y_pred = [1 if s >= tau else 0 for s in y_prob]
        TP, FP, TN, FN = confusion_counts(y_true, y_pred)
        TPR, FPR, TNR, PPV, NPV = metrics_from_counts(TP, FP, TN, FN)
        rows.append({
            "threshold": float(tau),
            "TP": TP, "FP": FP, "TN": TN, "FN": FN,
            "TPR": TPR, "FPR": FPR,
            "Recall": TPR, "Specificity": TNR,
            "Precision": PPV, "NPV": NPV,
            "Support_P": P, "Support_N": N
        })
    return rows

def roc_curve_points(roc_table):
    FPR = [row["FPR"] for row in roc_table]
    TPR = [row["TPR"] for row in roc_table]
    return FPR, TPR

def auc_trapezoid(x, y):
    pts = sorted(zip(x, y), key=lambda p: p[0])
    area = 0.0
    for (x0,y0),(x1,y1) in zip(pts[:-1], pts[1:]):
        dx = x1 - x0
        area += dx * (y0 + y1) / 2.0
    return area

def roc_auc_from_table(roc_table):
    FPR, TPR = roc_curve_points(roc_table)
```

```python
    return auc_trapezoid(FPR, TPR)

def plot_roc_curve(roc_table, title_prefix="ROC Curve (demo)"):
    FPR, TPR = roc_curve_points(roc_table)
    auc = roc_auc_from_table(roc_table)
    plt.figure(figsize=(6,6))
    plt.plot(FPR, TPR, marker='o', linewidth=1, label=f"Modelo (AUC={auc:.4f})")
    plt.plot([0,1], [0,1], linestyle='--', label="Azar (AUC=0.5)")
    plt.xlabel("FPR (False Positive Rate)")
    plt.ylabel("TPR (True Positive Rate / Recall)")
    plt.title(f"{title_prefix}     AUC={auc:.4f}")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
    return auc

# ---------- PR: tabla y AUPRC ----------
def compute_pr_table(y_true, y_prob, thresholds=None, add_endpoint_eps=True):
    if thresholds is None:
        thresholds = build_thresholds(y_prob, add_endpoint_eps=add_endpoint_eps)
    rows = []
    P = sum(1 for t in y_true if t == 1)
    for tau in thresholds:
        y_pred = [1 if s >= tau else 0 for s in y_prob]
        TP, FP, TN, FN = confusion_counts(y_true, y_pred)
        precision = (TP / (TP + FP)) if (TP + FP) else 1.0  # convenci n
        recall    = (TP / P) if P else 0.0
        rows.append({"threshold": float(tau), "TP":TP, "FP":FP, "TN":TN, "FN":FN,
                     "precision":precision, "recall":recall})
    rows.sort(key=lambda r: r["recall"])
    return rows

def pr_curve_points(pr_table):
    recall    = [r["recall"] for r in pr_table]
    precision = [r["precision"] for r in pr_table]
    return recall, precision

def pr_auc_from_table(pr_table):
    recall, precision = pr_curve_points(pr_table)
    return auc_trapezoid(recall, precision)

def plot_pr_curve(pr_table, title_prefix="Precision Recall Curve (demo)"):
    recall, precision = pr_curve_points(pr_table)
```

```python
    auprc = pr_auc_from_table(pr_table)
    pos_rate = recall[-1] if recall else 0.0  # baseline
    plt.figure(figsize=(6,6))
    plt.plot(recall, precision, marker='o', linewidth=1, label=f"Modelo (AUPRC={auprc:.4f})")
    plt.hlines(y=pos_rate, xmin=0, xmax=1, linestyles='--', label=f"Azar (pos rate {pos_rate:.3f})")
    plt.xlabel("Recall (TPR)")
    plt.ylabel("Precision (PPV)")
    plt.title(f"{title_prefix}     AUPRC={auprc:.4f}")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
    return auprc

# ---------- Helpers de impresi n en consola ----------
def print_roc_table(roc_table):
    print("\n=== TABLA ROC (todos los umbrales) ===")
    header = "Umbral   TP   FP   TN   FN   TPR    FPR    Prec   NPV"
    print(header)
    print("-"*len(header))
    mx = max(r["threshold"] for r in roc_table)
    mn = min(r["threshold"] for r in roc_table)
    for r in roc_table:
        t  = r["threshold"]
        # Etiquetas estilo >max / <min para los extremos
        if t > mx - 1e-12: th = f">{mx - 1e-12:.1f}"
        elif t < mn + 1e-12: th = f"<{mn + 1e-12:.1f}"
        else: th = f"{t:.1f}"
        print(f"{th:>6}  {r['TP']:>2}  {r['FP']:>2}  {r['TN']:>2}  {r['FN']:>2}  "
              f"{r['TPR']:>5.3f}  {r['FPR']:>5.3f}  {r['Precision']:>5.3f}  {r['NPV']:>5.3f}")

def print_pr_table(pr_table):
    print("\n=== TABLA PrecisionRecall (ordenada por recall) ===")
    header = "Umbral    TP   FP   TN   FN   Recall  Precision"
    print(header)
    print("-"*len(header))
    for r in pr_table:
        print(f"{r['threshold']:>7.3f}  {r['TP']:>2}  {r['FP']:>2}  {r['TN']:>2}  {r['FN']:>2}  "
              f"{r['recall']:>6.3f}   {r['precision']:>9.3f}")

# ---------- DEMO ----------
if __name__ == "__main__":
    # Datos ficticios (N = 0, P = 1)
    N = [0.3,0.4,0.5,0.5,0.5,0.6,0.7,0.7,0.8,0.9]
    P = [0.5,0.6,0.6,0.8,0.9,0.9,0.9,1.0,1.2,1.4]
```

```
    y_prob = N + P
    y_true = [0]*len(N) + [1]*len(P)

    # Tablas
    roc_table = compute_roc_table(y_true, y_prob,
        add_endpoint_eps=True)
    pr_table  = compute_pr_table (y_true, y_prob,
        add_endpoint_eps=True)

    # Impresi n en consola
    print_roc_table(roc_table)
    print_pr_table(pr_table)

    # AUCs + gr ficas (en ventana)
    auc_roc = plot_roc_curve(roc_table, title_prefix="ROC Curve (
        demo)")
    print(f"\nAUC(ROC) = {auc_roc:.6f}")
    auprc = plot_pr_curve(pr_table, title_prefix="
        P r e c i s i o n Recall  Curve (demo)")
    print(f"AUPRC    = {auprc:.6f}")
```

7 Mensajes clave para el aula

- La curva ROC **se construye contando**: cada fila de la tabla corresponde a una matriz de confusión en un umbral específico.
- Los *ties* (empates de score entre clases) provocan *saltos oblicuos* o pasos más largos.
- En presencia de desbalance, complemente ROC con **PR** y **AUPRC**.

ANEXO 1. Ubicación del Cálculo ROC en el Proceso de Entrenamiento e Inferencia

Dentro del ecosistema de NVIDIA, las herramientas **TAO Toolkit**, **DALI** y **TensorRT** cubren distintas etapas del ciclo de vida de un modelo de visión por computadora. El cálculo de la **Curva ROC** se utiliza como métrica de evaluación posterior al entrenamiento, para medir la capacidad discriminante del modelo antes de su despliegue final. La Figura **??** resume esta relación.

Dataset (DALI) $\longrightarrow$ Entrenamiento (TAO) $\longrightarrow$ Evaluación ROC (TAO / Python) $\longrightarrow$ Despliegue

NVIDIA DALI: Preprocesamiento y Aumento de Datos

El componente **DALI (Data Loading Library)** se utiliza durante la fase de entrenamiento para realizar operaciones de preprocesamiento y *data augmentation* directamente en la GPU. Esto incluye tareas como rotación, escalado, recorte y normalización de imágenes, lo que permite aumentar la robustez del modelo sin sobrecargar la CPU.

DALI no participa en el cálculo ROC, ya que su función se limita a preparar los datos de entrada.

NVIDIA TAO Toolkit: Entrenamiento y Evaluación del Modelo

El **TAO Toolkit** (Train, Adapt, Optimize) se encarga tanto del entrenamiento como de la evaluación del modelo. En esta etapa se generan los *scores* (probabilidades) que servirán para calcular la curva ROC y su correspondiente área bajo la curva (AUC). Durante el entrenamiento se utiliza un archivo de configuración `.yaml` que define la arquitectura (por ejemplo, `ResNet-50`) y los parámetros de optimización.

Listing 2: Entrenamiento con TAO Toolkit

```
# Entrenamiento de un modelo ResNet
tao classification train -e specs/resnet_spec.yaml \
    -r results/resnet_experiment \
    -k nvidia_tlt
```

Al finalizar el entrenamiento, se ejecuta el módulo de evaluación que genera las predicciones de probabilidad necesarias para construir la tabla ROC:

Listing 3: Evaluación y generación de scores

```
# Evaluaci n del modelo entrenado
tao classification evaluate \
    -m results/resnet_experiment/weights/resnet.tlt \
    -e specs/resnet_eval.yaml \
    -k nvidia_tlt
```

El resultado es un archivo con las etiquetas verdaderas y las probabilidades predichas, desde el cual puede calcularse la curva ROC mediante herramientas estándar de Python:

Listing 4: Cálculo de la Curva ROC y AUC

```
from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_true, y_scores)
auc_value = auc(fpr, tpr)
```

NVIDIA TensorRT: Inferencia Optimizada

Una vez validado el modelo, se procede a convertirlo y optimizarlo mediante **TensorRT**, que produce un motor de inferencia altamente eficiente para ejecución en dispositivos Jetson, GPU de escritorio o servidores con Triton Inference Server. En esta fase ya no se calcula la curva ROC; el objetivo es acelerar la inferencia con precisión y baja latencia.

Listing 5: Conversión del modelo a TensorRT

```
# Conversi n a motor TensorRT optimizado
tao converter --model resnet.etlt --key nvidia_tlt \
    --data_type fp16 --engine_file resnet.engine
```

Interpretación y Uso del Cálculo ROC

El cálculo ROC se realiza con los resultados generados por TAO y permite:

- Evaluar la capacidad discriminante del modelo entre clases positivas y negativas.
- Determinar el umbral óptimo t^* para balancear sensibilidad (TPR) y especificidad ($1 - FPR$).
- Comparar diferentes configuraciones o arquitecturas de modelos.

En entornos de control de calidad industrial, el umbral t^* suele elegirse de modo que se minimicen los falsos negativos, garantizando que ningún producto defectuoso pase inadvertido.

Resumen Conceptual

Herramienta	**Fase del proceso**	**Participación en ROC**
DALI	Preprocesamiento y augmentations	No, sólo prepara los datos
TAO Toolkit	Entrenamiento y evaluación	Sí, genera los *scores*
TensorRT	Despliegue e inferencia	No, sólo ejecuta el modelo

Table 3: Rol de cada herramienta NVIDIA en relación con la Curva ROC

En síntesis, el cálculo de la Curva ROC se inserta entre la etapa de evaluación del modelo en **TAO Toolkit** y la optimización para inferencia en **TensorRT**. Mientras DALI acelera la lectura y transformación de datos, TAO genera las métricas y TensorRT ejecuta el modelo final. El análisis ROC, por tanto, es un paso crítico de validación que informa las decisiones sobre umbrales y despliegue, asegurando que la red convolucional (por ejemplo, una ResNet) no sólo sea precisa en entrenamiento, sino también robusta y confiable en producción.

ANEXO 2. Uso de la Curva ROC en la Evaluación del Sistema AOI Basado en Deep Learning

Una vez entrenado el modelo de inspección visual (por ejemplo, una red **ResNet-50** dentro del entorno **NVIDIA TAO Toolkit**), es necesario evaluar su capacidad de discriminación entre piezas correctas y defectuosas. Para ello se utiliza la **Curva ROC (Receiver Operating Characteristic)**, que permite analizar el comportamiento del clasificador a través de todos los umbrales de decisión posibles.

Fundamento del método

El modelo genera una probabilidad $p \in [0, 1]$ que representa el grado de confianza en que una imagen corresponde a una pieza defectuosa. Si definimos un umbral t, la decisión se toma como:

$$\text{Defectuosa si } p \geq t, \quad \text{Correcta si } p < t.$$

Para cada valor de t, se construye una tabla de confusión y se calculan:

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}.$$

El conjunto de puntos (FPR, TPR) genera la curva ROC, y el área bajo la misma (AUC) mide la capacidad global del modelo para distinguir entre ambas clases.

Implementación dentro del flujo NVIDIA

1. **Preprocesamiento (DALI):** acelera la lectura y transformación de imágenes en GPU. No interviene directamente en el cálculo ROC, pero mejora la calidad del dataset.

2. **Entrenamiento (TAO Toolkit):** entrena la red y genera los valores de probabilidad (*scores*) asociados a cada imagen.

3. **Evaluación (TAO / Python):** se calculan las métricas ROC mediante el siguiente código:

   Listing 6: Cálculo de la Curva ROC y AUC

   ```
   from sklearn.metrics import roc_curve, auc

   fpr, tpr, thresholds = roc_curve(y_true, y_scores)
   auc_value = auc(fpr, tpr)
   ```

4. **Despliegue (TensorRT / Triton):** se selecciona el umbral óptimo t^* y el modelo se convierte a motor de inferencia. En esta fase ya no se calcula la ROC.

Selección del umbral óptimo

El umbral t^* puede definirse como el punto donde la diferencia $TPR - FPR$ es máxima (criterio de Youden):

$$t^* = \arg\max_t (TPR - FPR).$$

En el contexto de manufactura electrónica, suele preferirse un umbral que minimice los *falsos negativos*, garantizando que ningún defecto escape a la inspección, aunque se tolere un leve incremento de falsos positivos.

Importancia práctica

El análisis ROC permite cuantificar de forma objetiva el equilibrio entre **rendimiento** y **riesgo**:

- Un AUC cercano a 1.0 indica una excelente separación entre clases.
- Un AUC de 0.5 refleja un modelo aleatorio sin poder discriminante.
- Ajustar el umbral t según el perfil de riesgo de la planta permite optimizar el costo total de inspección.

El cálculo de la curva ROC es un componente esencial del ciclo de validación en sistemas AOI basados en *Deep Learning*. Permite establecer umbrales de decisión óptimos y cuantificar la calidad de la clasificación antes de transferir el modelo a **TensorRT** para inferencia en producción, asegurando así la máxima confiabilidad del sistema de inspección.

ANEXO 3. Aplicación del Deep Learning en Inspección Óptica Automatizada (AOI)

Limitaciones de los sistemas tradicionales basados en reglas

In summary, the current process faces several challenges:

- Traditional rules-based AOI machine produces high false positive rate, which decreases throughput and increases cost.
- Lacks flexibility to accommodate rapidly evolving parts and product designs.
- Cost of defect escape is very high.

Although AOI has eclipsed manual inspection in situations where complexity, throughput, or both, are high, traditional image-processing systems and algorithms have some drawbacks that are apparent in system and software development and when setting up equipment on the factory floor. To understand why automated optical inspection is difficult, consider the vast criteria that must be tested. When inspecting a PCBA, the quality of solder joints is only one criterion to inspect. The presence of each component must also be verified, as well as the position and orientation relative to the solder mask. Other criteria include component coplanarity, if the components are lifted, or if there is presence of unwanted objects such as solder spatter/ball or other contamination on the surface of the board. It is almost impossible to create rules for all cases and all exceptions. Deep learning-based solutions can be used to solve this problem.

Explicación y justificación del enfoque basado en *Deep Learning*

El texto anterior describe los límites de las máquinas de **Inspección Óptica Automatizada (AOI)** basadas en reglas, que utilizan filtros, umbrales e instrucciones predefinidas para detectar defectos en placas de circuito impreso (PCBA). Aunque efectivas en entornos controlados, estas soluciones presentan tres problemas principales:

1. **Alta tasa de falsos positivos**: pequeñas variaciones en iluminación, textura o ángulo provocan que el sistema detecte defectos inexistentes.
2. **Falta de flexibilidad**: cada cambio de componente o diseño requiere ajustar manualmente umbrales y parámetros.
3. **Costo de mantenimiento elevado**: reprogramar reglas y revisar falsos positivos reduce la eficiencia de producción.

Cómo el *Deep Learning* aborda estos desafíos

Los modelos de **aprendizaje profundo**, especialmente las redes neuronales convolucionales (CNNs), no dependen de reglas fijas, sino que *aprenden directamente de los datos* las características relevantes para distinguir entre uniones de soldadura correctas y defectuosas. Este enfoque presenta múltiples ventajas:

- **Aprendizaje de características complejas**: la red aprende formas, texturas, brillos y patrones de defectos sin intervención manual.
- **Adaptabilidad**: mediante técnicas como *transfer learning*, el modelo puede ajustarse a nuevos productos o variaciones de diseño sin reescribir código.
- **Reducción de falsos positivos y negativos**: el modelo aprende la variabilidad natural de la línea de producción, logrando un mejor balance entre sensibilidad y especificidad.

Integración con el ecosistema NVIDIA

El flujo de trabajo de NVIDIA para visión industrial permite implementar este enfoque de manera eficiente:

1. **NVIDIA DALI**: acelera el preprocesamiento y las transformaciones de datos en GPU.
2. **NVIDIA TAO Toolkit**: entrena modelos basados en arquitecturas como ResNet o EfficientNet, generando las probabilidades (*scores*) utilizadas para evaluar desempeño mediante curvas ROC.
3. **NVIDIA TensorRT**: optimiza el modelo final para inferencia en tiempo real, reduciendo latencia y aumentando throughput en la línea de producción.

Ejemplo práctico

En lugar de definir reglas como:

$$\text{Si la intensidad de píxel} < 120 \text{ en la región } X, \text{ entonces hay falta de soldadura}$$

el modelo de *Deep Learning* aprende directamente la función:

$$f(\text{imagen}) \rightarrow \text{probabilidad de defecto}$$

y puede clasificar por tipo: *puente, exceso de estaño, contaminación, mala alineación*, entre otros.

El enfoque basado en *Deep Learning* supera las limitaciones del AOI tradicional al eliminar la dependencia de reglas estáticas, aprender de ejemplos reales y adaptarse dinámicamente a las variaciones del entorno industrial. La combinación de **TAO Toolkit, DALI y TensorRT** dentro del ecosistema NVIDIA permite crear sistemas de inspección visual más robustos, escalables y precisos, capaces de reducir costos y mejorar la calidad en procesos de manufactura avanzada.

Bibliografía

W. J. Krzanowski and D. J. Hand, *ROC Curves for Continuous Data* Boca Raton, FL: Chapman and Hall/CRC, 2009.